V0 SPEC INPUT · MEMORY & INTERCONNECT · FOR DV

Coral Memory & Interconnect

Read from RTL at commit `9f31e14e`, 2026-10-10. "Shipped" means the flags in `hdl/chisel/src/coralnpu/BUILD` for `CoreMiniAxi` / `RvvCoreMiniAxi` / `VmeCoreMiniAxi`. Paths are relative to `hdl/chisel/src/` unless noted.

`STABLE` fixed in RTL, expect to carry over `LIKELY` current choice, low churn `VARIABLE` parameter or expected to change

`HIGH` / `MED` / `LOW` = deviation impact on DV plan

Summary `read-aloud version`

The idea. Coral's memory system is deliberately simple. There are no caches. The core has two private scratchpads that software manages, and AXI ports to everything else. Software decides what sits in the scratchpads; hardware never moves data on its own.

The TCMs. The ITCM holds code (8 KB by default). The DTCM holds data (32 KB by default). Both can be configured up to 1 MB. Each is a single-port SRAM, 128 bits (16 bytes) wide, with a 1-cycle read and byte write-enables. That means one access per cycle per TCM. The docs call them banked, but the RTL doesn't bank them.

The buses. Inside the core there are three buses, all 128 bits wide. The load/store unit picks one by address. The **ibus** fetches instructions from ITCM: one 16-byte row is four instructions per cycle, which is exactly what the 4-wide front end needs. Loads from ITCM share it. The **dbus** connects the load/store unit to DTCM. Scalar, vector and matrix all share this one port, so DTCM peaks at 16 bytes per cycle. The **ebus** carries everything else to a bridge that turns it into AXI.

CoreAxi. This is the whole Coral core treated as one black box. Inside is the processor itself: the RISC-V CPU with its vector and matrix units, which the other teams cover. Around it is our part: both TCMs, the control registers, and the local interconnect, which is a set of arbiters and an address decoder. The debug module is in there too. Our section is CoreAxi minus the processor, plus the SoC crossbar outside it. Outside are two AXI4 ports (32-bit address, 128-bit data) and a few sideband pins: clock, reset, boot address, three interrupts, halted/fault/wfi status, and debug. The **AXI master** is the core reaching out. Data uses ID 0, instruction fetch outside ITCM uses ID 1, and only one single-beat transaction can be outstanding per ID. The **AXI slave** is a host or DMA reaching in to load the TCMs and start the core. It handles one transaction at a time and supports bursts. The core always has priority on the TCMs, so inbound traffic can wait.

Everything else. Outside CoreAxi is the SoC: a TileLink crossbar that connects the core to a DMA engine, 4 MB of SRAM, DDR and peripherals. It's a reference FPGA design and the part most likely to change for our chip. Its DMA has one channel and one transfer in flight.

Why it matters for us. For an LLM chip the limit is bandwidth, not compute: 16 bytes per cycle into DTCM, and one outstanding transaction to DDR. For the spec, several docs disagree with

the RTL. The big three: the TCMs aren't banked, the AXI master uses two IDs, and there's no single crossbar.

At a glance

- **No caches.** Two software-managed scratchpads: ITCM (code) and DTCM (data). Every other address is uncached AXI.
- **128-bit everywhere inside the core.** Each TCM is single-port 128-bit SRAM with 16 byte enables and a fixed 1-cycle rea latency.
- **The LSU routes by address:** ITCM → `ibus` (reads only), DTCM → `dbus`, anything else → `ebus` → AXI master.
- **Scalar, RVV and VME share one LSU**, so all three share one 16 B/cycle DTCM port. Vector and matrix units have no mem port of their own.
- **The core boundary is an AXI4 master plus an AXI4 slave.** The TL-UL variant (`CoreTlul`) is the same `CoreAxi` with bridges added.

CoreAxi as a black box `CoreAxi.scala:25-46`

PIN	DIR	WIDTH	FUNCTION
`aclk`	in	1	The only clock; everything inside runs on it
`aresetn`	in	1	Active-low async reset, synchronized inside
`te`	in	1	Scan/test enable. Bypasses the reset synchronizer and th clock gate
`boot_addr`	in	32	Copied into PC_START on the first cycle after reset
`irq` , `timer_irq` , `software_irq`	in	1 each	Registered for 1 cycle at the boundary. Any of them wake the clock gate
`halted` , `fault` , `wfi`	out	1 each	Core status, also readable in STATUS
`axi_slave`	in	AXI4 a32 d128 id6	Host/DMA reads and writes to ITCM, DTCM, CSR
`axi_master`	out	AXI4 a32 d128 id6	Core loads, stores and fetches outside the TCMs
`dm.req / dm.rsp`	in / out	{addr32, data32, op2} / {data32, op2}	Debug module access (RISC-V debug spec 0.13.2). Shared round-robin with the CSR mailbox
`debug`	out	trace	Only in verification builds

Behavior at the pins

- **Boot.** After reset, the core is held in reset with its clock gated (RESET_CONTROL = 0x3). The host loads ITCM over `axi_slave` , optionally rewrites PC_START, clears the clock gate, then clears reset. Both AXI ports are held off for the f cycle after reset.
- **`axi_slave` .** One transaction at a time, in order. FIXED/INCR/WRAP bursts up to 256 beats. Returns OKAY for TCMs and valid CSRs, SLVERR for anything else. Latency has no upper bound, because the core has priority on each TCM.
- **`axi_master` .** ID 0 is data, ID 1 is fetch. Every transaction is a single beat. Data sizes are 1–16 B; fetch is 16 B and aligne

most one transaction is outstanding per ID, so at most 2 are in flight: one data read or write, plus one fetch. Responses the two IDs can return in either order.

- **Loopback.** When the core accesses its own CSR range, the access leaves on `axi_master`. The SoC must route it back `axi_slave`.
- **Errors.** A non-OKAY response on `axi_master` becomes a precise trap inside the core.

Quick reference: bit widths

SIGNAL / FIELD	BITS	NOTES
COREAXI PINS		
`aclk`, `aresetn`, `te`	1 each	aresetn active-low, async
`boot_addr`	32	Sampled once after reset
`irq`, `timer_irq`, `software_irq`	1 each	Level-sensitive. MEIP / MTIP / MSIP
`halted`, `fault`, `wfi`	1 each	halted and fault are sticky until core re
AXI4, BOTH PORTS		
addr (AW, AR)	32	
data (W, R)	128	
strb (W)	16	1 per byte
id (all channels)	6	Master uses 0 (data) and 1 (fetch)
len	8	Master always 0. Slave accepts up to 25
size	3	0–4 = 1–16 B
burst	2	Master INCR. Slave accepts FIXED/INCR WRAP
prot / cache / qos / region / lock	3/4/4/4/1	Master drives constants (prot = 2). Slav ignores them
resp (B, R)	2	OKAY or SLVERR; never EXOKAY
last (W, R)	1	No user signals
DEBUG PORT		
`dm.req` address / data / op	32/32/2	op: NOP, READ, WRITE
`dm.rsp` data / op	32/2	op: SUCCESS, FAILED, BUSY
INTERNAL (WHITE-BOX)		
`ibus`	addr 32 · data 128	One 16 B row = 4 instructions
`dbus`	addr 32 · data 128 · mask 16 · size 5	size is a power-of-two byte count

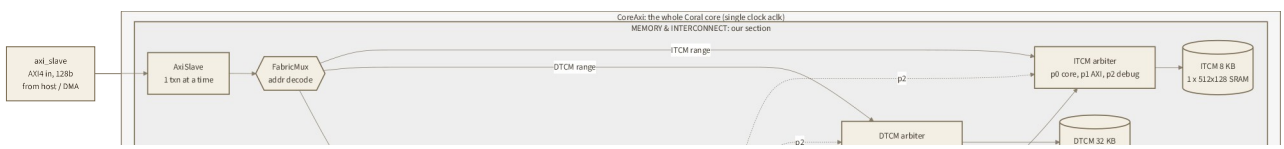
FabricIO	addr 32 · data 128 · strb 16	No ready and no ID
SRAM macro	addr 9 · data 128 · mask 16	Per 512×128 macro
CSR register	32	On the 128 b bus, lane = (offset mod 16

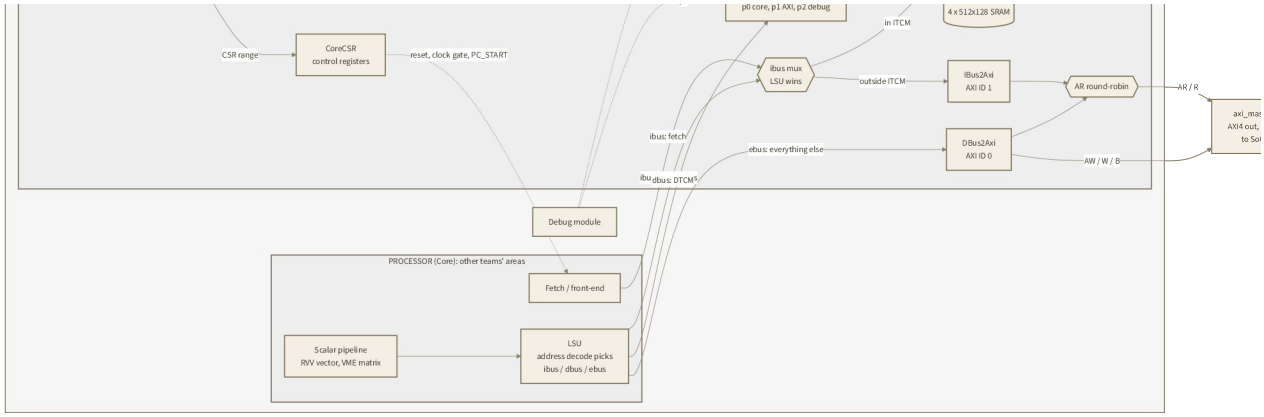
ITCM & DTCM `CoreAxi.scala:152-237`

	ITCM	DTCM
Holds	Code. The core can also load from it	Data for scalar, RVV and VME
Default base / size	0x0000_0000 / 8 KB	0x0001_0000 / 32 KB
Custom base / max	0x0000_0000 / 1 MB	0x0010_0000 / 1 MB
Row width	128 b = 16 B. All access is in rows	
Rows (default)	512, row = addr[12:4]	2048, row = addr[14:4]
Physical (default)	1 × 512×128 macro	4 × 512×128 macros, depth-stacked. addr[14:13] picks the macro
Physical (≥64 KB)	4096-entry blocks have no ASIC macro and hit $error . FPGA or simulation only	
Ports	Single-port: 1 read *or* 1 write per cycle	
Read latency	1 cycle, fixed	
Write granularity	Byte (16-bit mask)	
Requesters, in priority order	① core ibus (fetch; LSU loads win over fetch) ② axi_slave ③ debug	① core dbus (LSU: scalar, RVV, VME) ② axi_slave ③ debug
Core access	Read only; a store faults	Read/write, always a full row
Host access	Read/write over axi_slave , with bursts	
Error path	Core store → access fault	None
Clock / reset	Always-on clock, so it keeps running while the core is gated. Contents are not reset	
Peak bandwidth	16 B/cycle = 4 instr/cycle, enough for 4-wide dispatch	16 B/cycle, shared by the LSU and host

Not "4 × 32-bit banks". Each row is one 128-bit word at one address. Bus, row and macro are all 128 bits, so nothing is converted inside core. An ITCM row holds four 32-bit instructions, but they're read together and split after the read. The DTCM's 4 macros are stacked by depth, so only one is active per access. Concurrency would need address-interleaved banks (BankedDtcm.scala), which aren't instantiated.

Inside the box: processor + memory system `CoreAxi.scala internals`





There are two protocol layers. Inside the core, `FabricIO` carries a valid and an address, has no ready (backpressure is a side-band bu signal), uses no IDs or bursts, and returns read data exactly one cycle later. Outside the core it's AXI4. Arbiters are fixed-priority: the core always wins, so DMA/host traffic waits whenever the LSU is busy.

There is a processor inside CoreAxi. `CoreAxi` is the whole Coral core. The processor (`Core` : fetch, scalar pipeline, FPU, RVV, VME, LS sits inside it next to the memory system. Coral's RTL has no separate memory-system module; the memory system is the logic around `Core` . For our chip's spec we cut it out as its own black box:

FACES	PORT	INTO MEMORY SUBSYSTEM	OUT OF MEMORY SUBSYSTEM	WIDTHS
Processor	`ibus`	valid, addr	ready, rdata, fault	`addr 32 · data 128`
Processor	`dbus`	valid, write, addr, wdata, wmask, size, pc	ready (always 1), rdata	`addr 32 · data 128 · mask`
Processor	`ebus`	Same as dbus, exact size; internal	ready, rdata, fault	`size 1–16 B`
Processor	control	halted, fault (into STATUS)	core reset, clock gate, PC_START	`1 · 1 · 32`
SoC	`axi_slave`	AXI4 requests from host/ DMA	AXI4 responses	`a32 · d128 · id6`
SoC	`axi_master`	AXI4 responses	AXI4 requests to SoC	`a32 · d128 · id6`
Debug	TCM access	Debug module reads/ writes (arbiter port 2)	read data	`FabricIO`

Key parameters

PARAMETER	SHIPPED	RANGE / NOTE	
Data bus (LSU, fetch, AXI)	128 b	Class default 256, BUILD overrides to 128. TCM is hard-coded 128 (`TCM.scala:27`)	STAB
ITCM size	8 KB	8 KB – 1 MB (non-default sizes switch to the highmem map)	VAR
DTCM size	32 KB	32 KB – 1 MB	VAR
SRAM macro	512×128 SP	Blocks chosen from 4096/512/128 entries. Hard macros exist only for 512 and 2048 entries (TSMC12, GF12, GF22). 2048 is never selected. Power-of-two TCMs of 64 KB or more use 4096-entry blocks, which hit `$error` in every ASIC flow (FPGA or	LIKE

		simulation only)	
TCM read latency	1 cycle	Fixed. All internal muxing assumes it	STAB
TCM access rate	1 / cycle	3 requesters, fixed priority: core > AXI slave > debug	STAB
AXI ID width	6 b	`axi2IdBits` . Widened by the SoC crossbar	LIKE
AXI master outstanding	1 per ID	Single beat (`len=0`). Blocks until the response returns	VAR
AXI slave	1 txn	AR/AW round-robin. FIXED/INCR/WRAP bursts up to 256 beats	LIKE
Clock / reset	1 clk	Async-assert reset synchronized by `RstSync` . Core clock gated on WFI or by the CSR	LIKE

Core memory map `Parameters.scala:43`

REGION	DEFAULT	HIGHMEM / CUSTOM
ITCM	`0x0000_0000 +8 KB`	`0x0000_0000 +≤1 MB`
DTCM	`0x0001_0000 +32 KB`	`0x0010_0000 +≤1 MB`
CSR	`0x0003_0000 +4 KB`	`0x0020_0000 +4 KB`
Anything else	→ `ebus` → AXI master (ID 0); fetch → IBus2Axi (ID 1)	

The LSU and `axi_slave` see the same addresses. An LSU write to ITCM faults; a host write to ITCM is allowed. `FabricMux` strips th base with `addr & ~base` , so each region must be aligned to its size.

CSR block `CoreAxiCSR.scala`

OFFSET	REGISTER	RESET
`0x000`	RESET_CONTROL: [0] reset, [1] clock gate (RW)	`0x3`
`0x004`	PC_START (RW)	`boot_addr` pin
`0x008`	STATUS: [0] halted, [1] fault (RO)	`0`
`0x100–0x120`	9 read-only mirrors of core CSRs	–
`0x800–0x814`	Debug mailbox: REQ_ADDR/DATA/OP, RSP_DATA/OP, STATUS	`0`
other	SLVERR	

Inside the box: internal interfaces `white-box, for assertions`

INTERFACE	FROM → TO	HANDSHAKE	PAYLOAD	RULES DV SHOULD CHECK
`ibus`	Fetch / LSU → ITCM or IBus2Axi	valid/ready; rdata on the next cycle	addr 32, rdata 128, fault	addr held stable until ready. LSU preempts fetch (`SCore.scala:56`

`dbus`	LSU → DTCM arbiter p0	valid/ready (ready tied 1)	addr, write, wdata 128, wmask 16, pc; rdata 128 on the next cycle	Always a full 16 B aligned row. No fa path
`ebus`	LSU → DBus2Axi	valid/ready + fault, internal	Exact size 1/2/4/8/16 B	Blocks until the AXI response. A non OKAY resp becomes a precise trap
`FabricIO`	AxiSlave, FabricMux, arbiters, SRAM, CSR	valid only, plus busy side-band	read addr / write addr+data+strb; readData, writeResp	Read data exactly 1 cycle later. Neve read and write in the same cycle (assert)
`axi_master`	Core → SoC	AXI4	ID 0 data, ID 1 fetch	R beats steered by ID. 2-deep skid o each channel. AW/W carry data only
`axi_slave`	SoC → core	AXI4	Bursts to ITCM, DTCM, CSR	One transaction at a time. SLVERR fo unmapped addresses or read-only C writes
`rvv2lsu` / `vme2lsu`	RVV / VME → LSU	Decoupled	Store data, indices, masks	Only lane 0 of the 2 is wired

SoC level `soc/CrossbarConfig.scala`

- A reference FPGA SoC. TL-UL crossbar with a 128-bit common width: a 1→N socket per host and an M→1 socket per de plus width bridges and async FIFOs where a port differs.
- **Hosts:** core 128, spi2tlul 128, dma 128, ISP ×2 at 64 (ISP clock), autoboot 32.
- **Devices:** core window @0x0, ROM 32 KB @`0x1000_0000`, SRAM 4 MB @`0x2000_0000`, CLINT, PLIC, UART/SPI/GPIO/I2C, CSRs @`0x4000_0000`+, ISP, DDR 2 GB @`0x8000_0000`. DDR is reached through a 128→256 bridge and a 256-bit AXI por the DDR clock domain.
- **DMA** (`bus/DmaEngine.scala`): one channel, linked-list descriptors, 1 outstanding transaction (read one beat, write o beat), no interrupt.
- Everything here is `VARIABLE` for our chip.

Perf quick numbers

≤16 B/cyc
Total DTCM bandwidth, shared by the LSU and inbound DMA (core wins)

16 B / RTT
Core load from DDR (1 outstanding). At a 100-cycle round trip ≈ 0.16 B/cycle

16 B / 2·RTT
DMA DDR→DTCM: read one beat, ther write one beat

≤16 MAC/cyc
int8 GEMV fed from DTCM (1 byte per MAC), however much compute there is